New York University Tandon School of Engineering
Computer Science and Engineering

CS-GY 6033: Written Homework 6.
Due Thursday, April 23, 2026, 11:59pm.

Collaboration is allowed on this problem set, but solutions must be written-up individually. If we suspect that you copied your solution directly from AI, we will set up a meeting with you where you will need to explain your solution. If you fail to do so, you will receive zero points for the homework.

Problem 1

In a file named `make_change.py`, create a function `make_change(t)` which computes the **number** of ways of combining American quarters (worth 25 cents), dimes (10 cents), nickels (5 cents), and pennies (1 cent) to make t cents.

It may be possible that there is no way to make change for the target, t, in which case your function should return 0. You should assume that your are very rich and have an unlimited number of each type of coin.

For example: `make_change(11)` should be 4, since we can either use:

- 11 pennies,
- 1 nickel and 6 pennies, or
- 1 dime and 1 penny, or
- 2 nickels and 1 penny.

There is starter code here, and you should submit your solution to the autograder.

Note: there is no target time complexity for this problem, but the autograder will time out if your solution takes more than 60 seconds to run. However, as long as you're not implementing a brute force solution, you should be fine.

Problem 2

Consider the same problem of computing the number of ways to make change, but now we are not rich – we only have a certain number of quarters, dimes, nickels, and pennies.

In `constrained_make_change.py`, write a function named `constrained_make_change(t, coins)`, where t is the target sum that we are trying to reach, and `coins` is a list of four elements containing the number of quarters, dimes, nickels, and pennies we have. The function should return then number of ways of adding up to t using only the coins we have.

For example, `constrained_make_change(25, [2, 0, 0, 25])` should return 2, since there are only two ways with the coins we have:

- 1 quarter, or
- 25 pennies.

There is starter code here, and you should submit your solution to the autograder.

Note: there is no target time complexity for this problem, but the autograder will time out if your solution takes more than 60 seconds to run. However, as long as you're not implementing a brute force solution, you should be fine.

Problem 3

Consider the following set of activities.

Activity #	Start	Finish	Weight
0	0	3	12
1	1	2.5	10
2	2	3	22
3	2.75	4.25	14
4	4	8	33
5	4.5	11.5	11
6	5	6	12
7	5.5	6.5	7
8	8	10	5
9	9	12	19

Suppose the dynamic programming solution to the weighted activity selection problem is run on this data, and let `cache` be an array storing the memoized solution to each subproblem. In particular, `cache[i]` is the weight of the max weight schedule considering only the activities $i, i+1, \ldots, 9$. Note that we will consider two activities `x` and `y` to be compatible if `y.start` $\geq$ `x.finish` or `x.start` $\geq$ `y.finish`.

1. What are the entries of `cache`? You do not need to include the "dummy" entry of zero at the end of the array.

   Hint: when checking your work, you should ask yourself: is it possible that `cache[i] < cache[i+1]`?

2. What is the weight of the optimal solution to the weighted activity selection problem for this data?

   Hint: the right answer is greater than 70, less than 80.

Problem 4

In lecture, we designed a dynamic programming solution for the weighted activity scheduling problem. Our solution computed the *weight* of the best possible schedule, but did not return the schedule itself. Luckily, an optimal schedule can be computed using the `cache` array as long as we know for each event i, the *next* event j which begins on or after i's finish time.

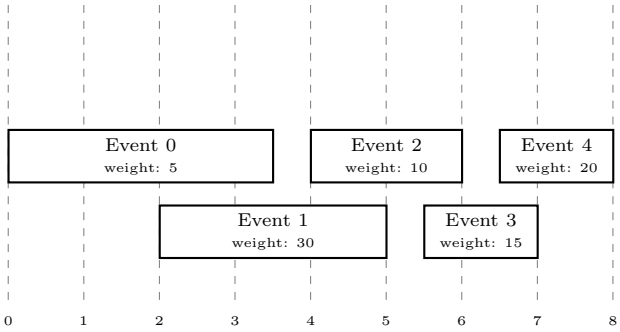
In a file named `recover_schedule.py`, write a function `recover_schedule(cache, next_activity)` which accepts two arguments:

- `cache`: The cache array as computed by the dynamic programming solution. It will be a Python list of numbers, where `cache[i]` is the weight of the best schedule that can be made from activities $i, i+1, \ldots, n-1$. `cache` *will* have one dummy entry of zero at the end, so that `len(cache)` is $n+1$, where n is the number of activities.
- `next_activity`: A list of n integers where `next_activity[i]` is the first activity starting after activity i finishes. We assume that the first activity is numbered zero, and that activities are sorted in order of start time. If there is no next activity possible, this entry should equal n.

Your function should return a list of integers representing the events that are in the optimal schedule. For instance, the list `[0, 2, 3]` represents the schedule consisting of the events 0, 2, and 3. There may be more than one optimal schedule; you need only return one of them.

Example: Consider the instance of the weighted scheduling problem shown below:

Activity #	Start	Finish	Weight
0	0	3.5	5
1	2	5	30
2	4	6	10
3	5.5	7	15
4	6.5	8	20



The best solution is to do activities 1 and 4, for a total weight of 50 (check for yourself!)

The dynamic programming `cache` will contain `[50, 50, 30, 20, 20, 0]`, where `cache[i]` represents the best solution using only activities from the set $\{i, i+1, \ldots, n-1\}$. The last entry of zero is the dummy entry mentioned above.

The `next_activity` list contains `[2, 3, 4, 5, 5]`. The 5s at the end denote that there is no possible next activity for Events 3 and 4.

Therefore, your code should behave as follows:

```
>>> cache = [50, 50, 30, 20, 20, 0]
>>> next_activity = [2, 3, 4, 5, 5]
>>> recover_schedule(cache, next_activity)
[1, 4]
```

Note that the code does not need to know anything about the actual activities, such as their weight – it only needs to know what the *next* activity is.

Hint: the hard part of this problem isn't writing the code – it is determining how to go from the `cache` to a schedule. Before writing any code, try to figure out the right strategy using a small example.

Problem 5

Here is a common interview problem. You are given a set of `valid_words` and a string, `s`. Your goal is to count the maximum number of matches of a word in `valid_words` in `s` such that no two matches overlap.

For example, suppose

```
valid_words = {"banana", "an", "ana", "ice", "cream"}
```

and

```
s = "icedbanana".
```

One way to count non-overlapping matches is to see that `"ice"` and `"banana"` both appear once, for a total of two matches. Similarly, we can match `"ice"` and `"ana"` once each without overlaps (if we allowed overlaps, `"ana"` would match twice). But neither of these are optimal. The most non-overlapping counts can be achieved by matching `"an"` twice and `"ice"` once, for a total of three matches.

The code below implements an inefficient (exponential time) algorithm for counting the number of non-overlapping matches:

```
def count_non_overlapping(s, valid_words, start=0):
    """Count number of non-overlapping matches.

    Parameters
    ----------
    s : str
        The string to find matches in.
    valid_words : Set[str]
        A set of strings that should be considered valid words.
    start : int
```

```
    An index into `s`. The substring s[start:] will be searched.

Returns
-------
count
    Maximum number of non-overlapping matches.

"""
best = 0
for stop in range(start+1, len(s) + 1):
    is_word = int(s[start:stop] in valid_words)
    n = is_word + count_non_overlapping(s, valid_words, stop)
    best = max(best, n)
return best
```

In a file named `count_non_overlapping.py`, create a

```
count_non_overlapping_td(s, valid_words, start)
```

which implements an top-down dynamic programming solution for this problem. Your code should be a modification of the code above, and it should run in time polynomial in the length of the list.